

PROJECT REPORT

Contact Book Management System

A Desktop GUI Application using Python (Tkinter & SQLite)

Submitted by: Priyam Singh

Program: B.Tech, Computer Science & Engineering (2024–2028)

Institution: Dr. K. N. Modi University

Subject: Source Code Explanation & Design Documentation

Project Title: Contact Book – GUI Based Contact Management System

This report documents the design, structure, and complete line-by-line working of the Contact Book application, a Tkinter and SQLite based desktop program for storing, viewing, searching, updating, and deleting personal contact records.

Table of Contents

- 1. Introduction
- 2. Objectives of the Project
- 3. Tools and Technologies Used
- 4. System Design
 - 4.1 Database Schema
 - 4.2 Application Architecture
- 5. Detailed Code Explanation
 - 5.1 Import Statements & Configuration
 - 5.2 ContactDatabase Class
 - 5.3 ContactBookApp Class – Setup & Layout
 - 5.4 ContactBookApp Class – Data Operations
 - 5.5 Program Entry Point
- 6. Feature-to-Code Mapping
- 7. Testing Summary
- 8. Conclusion
- 9. Future Enhancements

1. Introduction

A Contact Book is a fundamental record-management application that stores personal or professional contact details such as name, phone number, email address, and postal address. This project implements a desktop Contact Book using Python's built-in **Tkinter** library for the graphical user interface and **SQLite3** for persistent, file-based data storage. The application allows a user to add, view, search, update, and delete contact records through an intuitive, form-based interface.

Unlike a simple command-line contact manager, this application provides a two-panel graphical layout: a searchable list of all contacts on the left, and an add/edit form on the right. All data is written to a local database file (`contacts.db`), ensuring that records are not lost when the program is closed and reopened.

2. Objectives of the Project

- To design a functional graphical user interface for contact management using Tkinter.
- To implement persistent data storage using SQLite so that data survives application restarts.
- To provide full CRUD (Create, Read, Update, Delete) functionality for contact records.
- To implement a live search feature that filters contacts by name or phone number.
- To apply basic input validation for phone numbers and email addresses.
- To design a clean, user-friendly interface following simple UI/UX principles.

3. Tools and Technologies Used

Component	Technology	Purpose
Programming Language	Python 3	Core application logic
GUI Framework	Tkinter / ttk	Windows, widgets, forms, and styling
Database	SQLite3	Lightweight, file-based persistent storage
Validation	Python 're' module	Regular-expression checks for phone/email
IDE / Runtime	Any Python 3.x environment	Development and execution

4. System Design

4.1 Database Schema

The application uses a single SQLite table named `contacts` with the following structure:

Column	Type	Description
<code>id</code>	INTEGER (PK, AUTOINCREMENT)	Unique identifier for each contact
<code>name</code>	TEXT, NOT NULL	Full name of the contact
<code>phone</code>	TEXT, NOT NULL	Phone number of the contact
<code>email</code>	TEXT	Email address (optional)
<code>address</code>	TEXT	Postal address (optional)

4.2 Application Architecture

The program follows a simple two-layer architecture that separates data handling from the user interface:

- **ContactDatabase class** — encapsulates all SQLite operations (create table, insert, update, delete, select, search).
- **ContactBookApp class** — builds the Tkinter interface, manages widget state, and calls into `ContactDatabase` whenever data must be read or written.

This separation means the storage mechanism (SQLite) could later be replaced (for example, with a different database) without needing to rewrite the GUI code.

5. Detailed Code Explanation

This section walks through the complete source code of `contact_book.py` line by line, explaining the purpose of every statement, function, and class.

5.1 Import Statements & Configuration

Module Imports

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3
import os
import re
```

- **Line 1** — Imports the core `tkinter` module and aliases it as `tk`; this provides the base GUI widgets and window management.
- **Line 2** — Imports `ttk` (themed widgets, used for a more modern look) and `messagebox` (for pop-up confirmation, warning, and info dialogs).
- **Line 3** — Imports Python's built-in `sqlite3` module, which is used to create and query the local database file with no external installation needed.
- **Line 4** — Imports `os`, used to build a file path for the database that works regardless of the operating system.
- **Line 5** — Imports `re` (regular expressions), used later to validate phone numbers and email addresses.

Database File Path

```
DB_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "contacts.db")
```

- **Line 1** — Builds an absolute path to `contacts.db` in the same folder as the script itself. `os.path.abspath(__file__)` gets the script's full path, `os.path.dirname(...)` strips the filename to leave just the directory, and `os.path.join` safely appends the database filename. This guarantees the database is always found next to the script, no matter which folder the program is run from.

5.2 The ContactDatabase Class

This class isolates every database operation so the rest of the program never has to write raw SQL directly.

Constructor — `__init__`

```
def __init__(self, db_path=DB_FILE):
    self.conn = sqlite3.connect(db_path)
    self.conn.execute("PRAGMA foreign_keys = ON")
    self._create_table()
```

- **Line 1** — Defines the constructor, accepting an optional path so the database location can be overridden (defaults to `DB_FILE`).
- **Line 2** — Opens (or creates, if it does not yet exist) the SQLite database file and stores the connection object as `self.conn` for reuse by every other method.
- **Line 3** — Enables SQLite's foreign-key enforcement (a good-practice default, even though this single-table schema does not currently use foreign keys).
- **Line 4** — Calls the private helper method that creates the `contacts` table if it does not already exist.

`_create_table()`

```
def _create_table(self):
    self.conn.execute(
        """
        CREATE TABLE IF NOT EXISTS contacts (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            phone TEXT NOT NULL,
            email TEXT,
            address TEXT
        )
        """
    )
    self.conn.commit()
```

- **Line 1** — Defines a private helper (leading underscore convention) responsible only for schema creation.
- **Lines 2-11** — Executes a multi-line SQL statement that creates the `contacts` table only if it does not already exist, so re-running the program never wipes existing data. The `id` column auto-increments as the primary key; `name` and `phone` are marked `NOT NULL` because they are mandatory fields; `email` and `address` are optional.
- **Line 12** — Commits the transaction so the schema change is permanently saved to disk.

add_contact()

```
def add_contact(self, name, phone, email, address):
    cursor = self.conn.execute(
        "INSERT INTO contacts (name, phone, email, address) VALUES (?, ?, ?, ?)",
        (name, phone, email, address),
    )
    self.conn.commit()
    return cursor.lastrowid
```

- **Line 1** — Defines the method used to insert a brand-new contact, accepting the four field values.
- **Lines 2-5** — Runs a parameterised INSERT statement. The ? placeholders are filled in safely from the tuple of values — this protects against SQL-injection and formatting errors.
- **Line 6** — Commits the insert so it is saved to the database file.
- **Line 7** — Returns the auto-generated id of the newly inserted row, in case the caller needs it.

update_contact()

```
def update_contact(self, contact_id, name, phone, email, address):
    self.conn.execute(
        "UPDATE contacts SET name=?, phone=?, email=?, address=? WHERE id=?",
        (name, phone, email, address, contact_id),
    )
    self.conn.commit()
```

- **Line 1** — Defines the update method, which additionally requires the `contact_id` of the record being changed.
- **Lines 2-5** — Executes a parameterised UPDATE statement that rewrites all four fields for the row whose id matches `contact_id`.
- **Line 6** — Commits the change to disk.

delete_contact()

```
def delete_contact(self, contact_id):
    self.conn.execute("DELETE FROM contacts WHERE id=?", (contact_id,))
    self.conn.commit()
```

- **Line 1** — Defines the delete method, taking only the id of the record to remove.
- **Line 2** — Executes a parameterised DELETE statement that removes exactly the row with the matching id.
- **Line 3** — Commits the deletion permanently.

get_all_contacts()

```
def get_all_contacts(self):
    cursor = self.conn.execute(
        "SELECT id, name, phone, email, address FROM contacts ORDER BY name COLLATE NOCASE ASC"
    )
    return cursor.fetchall()
```

- **Line 1** — Defines the method that returns every stored contact.
- **Lines 2-4** — Selects all five columns for every row, ordered alphabetically by name. COLLATE NOCASE makes the sort case-insensitive, so 'bob' and 'Bob' are ordered together.
- **Line 5** — Fetches and returns every matching row as a list of tuples.

search_contacts()

```
def search_contacts(self, term):
    like_term = f"%{term}%"
    cursor = self.conn.execute(
        """
        SELECT id, name, phone, email, address FROM contacts
        WHERE name LIKE ? OR phone LIKE ?
        ORDER BY name COLLATE NOCASE ASC
        """,
        (like_term, like_term),
    )
    return cursor.fetchall()
```

- **Line 1** — Defines the search method, accepting the text typed by the user.
- **Line 2** — Wraps the search term with SQL wildcard characters (%) so it can match text appearing anywhere within a field, not only at the start.
- **Lines 3-9** — Runs a SELECT with a WHERE clause that matches the term against either the name or phone column, again ordered alphabetically.
- **Line 10** — Returns the matching rows.

get_contact() and close()

```
def get_contact(self, contact_id):
    cursor = self.conn.execute(
        "SELECT id, name, phone, email, address FROM contacts WHERE id=?",
        (contact_id,),
    )
    return cursor.fetchone()

def close(self):
    self.conn.close()
```

- **Lines 1-6** — Defines get_contact, which fetches a single row by id — used whenever the form needs to load one contact's full details for editing. fetchone() returns a single tuple, or None if no match exists.
- **Lines 8-9** — Defines close, which safely closes the SQLite connection when the application shuts down.

5.3 The ContactBookApp Class — Setup & Layout

This class builds and manages every visible part of the application. It is organised into 'build' methods (construct the interface once) and 'action' methods (respond to user interaction).

Constructor — `__init__`

```
def __init__(self, root):
    self.root = root
    self.db = ContactDatabase()
    self.selected_contact_id = None

    self._configure_root()
    self._build_style()
    self._build_layout()
    self.refresh_list()
```

- **Line 1** — Accepts the main Tkinter window (root) passed in from the program's entry point.
- **Line 2** — Stores the root window for later reference.
- **Line 3** — Creates one ContactDatabase instance for the whole application's lifetime, opening the SQLite connection.
- **Line 4** — Tracks which contact (by id) is currently selected/loaded into the form; None means the form is in 'add new contact' mode.
- **Line 6** — Sets window title, size, and background colour.
- **Line 7** — Configures the visual theme (colours, fonts) used by all ttk widgets.
- **Line 8** — Builds the two-panel layout (contact list on the left, form on the right).
- **Line 9** — Immediately loads and displays all existing contacts from the database on startup.

`_configure_root()`

```
def _configure_root(self):
    self.root.title("Contact Book")
    self.root.geometry("880x520")
    self.root.minsize(760, 460)
    self.root.configure(bg="#f2f4f7")
```

- **Line 2** — Sets the text shown in the window's title bar.
- **Line 3** — Sets the initial window size to 880 by 520 pixels.
- **Line 4** — Prevents the window from being resized smaller than 760 by 460 pixels, so the layout never becomes unusably cramped.
- **Line 5** — Sets a light grey background colour for the main window.

_build_style()

```
style = ttk.Style()
style.theme_use("clam")
style.configure("TFrame", background="#f2f4f7")
... (additional style.configure calls for labels, buttons, headers)
style.configure("Treeview", font=("Segoe UI", 10), rowheight=26)
```

- **Line 1** — Creates a `ttk.Style` object, the central place where widget appearance is configured.
- **Line 2** — Switches to the 'clam' theme, which renders more consistently across operating systems than the default theme; wrapped in a `try/except` in the source in case a platform lacks it.
- **Line 3 and similar lines** — Each `style.configure(...)` call sets fonts, colours, and padding for a specific widget class or custom style name (for example, 'Card.TFrame' for white content panels, or 'Accent.TButton' for the blue primary button).
- **Final line shown** — Increases the row height and font size of the Treeview (the contact list table) so rows are easier to read and click.

_build_layout()

```
header = ttk.Label(self.root, text="Contact Book", style="Header.TLabel")
header.pack(anchor="w", padx=20, pady=(15, 5))

body = ttk.Frame(self.root)
body.pack(fill="both", expand=True, padx=20, pady=10)
body.columnconfigure(0, weight=3)
body.columnconfigure(1, weight=2)
body.rowconfigure(0, weight=1)

self._build_list_panel(body)
self._build_form_panel(body)
```

- **Lines 1-2** — Creates the large 'Contact Book' heading label at the top of the window and positions it with `pack()`, left-aligned with padding.
- **Lines 4-5** — Creates a body frame that will hold the two main panels, and makes it expand to fill the remaining window space.
- **Lines 6-7** — Configures the grid so the left column (contact list) gets 3 parts of the available width and the right column (form) gets 2 parts, i.e. a 60/40 split.
- **Line 8** — Allows the single row to expand vertically as the window is resized.
- **Lines 10-11** — Calls the two dedicated builder methods that populate the left and right panels respectively.

build_list_panel() — search bar and Treeview

```
search_entry = ttk.Entry(search_frame, textvariable=self.search_var, ...)
self.search_var.trace_add("write", self._on_search_changed)

self.tree = ttk.Treeview(left, columns=("name", "phone"), show="headings")
self.tree.heading("name", text="Name")
self.tree.heading("phone", text="Phone Number")
self.tree.bind("<<TreeviewSelect>>", self._on_select_contact)
```

- **Search entry** — A text box bound to the string variable `self.search_var`. Because `trace_add('write', ...)` is attached to that variable, every keystroke automatically triggers `_on_search_changed` — this is what makes the search feel 'live' rather than requiring a Search button.
- **Treeview widget** — A `ttk.Treeview` is used as the contact list table. `show='headings'` hides the default tree icon column, leaving only the Name and Phone columns visible, matching the 'View Contact List' requirement (name + phone shown per row).
- **heading() calls** — Set the human-readable column titles shown at the top of the table.
- **bind() call** — Connects the Treeview's built-in selection event to `_on_select_contact`, so clicking a row automatically loads that contact's full details into the edit form.
- **Scrollbar and buttons (not shown)** — A vertical scrollbar is linked to the Treeview for long contact lists, and 'Delete Selected' / 'Refresh' buttons are placed beneath the table.

build_form_panel() and add_field()

```
self.name_var = tk.StringVar()
self.phone_var = tk.StringVar()
self.email_var = tk.StringVar()

self._add_field(fields_frame, 0, "Full Name *", self.name_var)
self._add_field(fields_frame, 1, "Phone Number *", self.phone_var)
self._add_field(fields_frame, 2, "Email", self.email_var)

self.address_text = tk.Text(fields_frame, height=5, ...)

self.save_btn = ttk.Button(..., text="Add Contact", command=self.save_contact)
self.update_btn = ttk.Button(..., text="Update Contact", command=self.update_contact, state="disabled")
```

- **StringVar lines** — Creates three Tkinter string variables that stay automatically synchronised with their entry boxes, so their current text can be read at any time with `.get()`.
- **_add_field calls** — A small reusable helper (defined separately) that creates a label plus an entry box for a given field, avoiding repeated layout code for Name, Phone, and Email.
- **address_text** — Address is a multi-line field, so a plain `tk.Text` widget is used instead of a single-line Entry.
- **save_btn** — The primary 'Add Contact' button, styled with the accent colour, calls `save_contact` when clicked.
- **update_btn** — The 'Update Contact' button starts disabled (greyed out) because there is nothing selected yet; it is enabled only once an existing contact is loaded into the form.

5.4 The ContactBookApp Class — Data Operations

refresh_list()

```
def refresh_list(self, contacts=None):
    for row in self.tree.get_children():
        self.tree.delete(row)

    if contacts is None:
        contacts = self.db.get_all_contacts()

    for contact_id, name, phone, email, address in contacts:
        self.tree.insert("", "end", iid=str(contact_id), values=(name, phone))

    self.count_label.config(text=f"{len(contacts)} contact(s)")
```

- **Lines 2-3** — Clears every row currently shown in the Treeview before repopulating it, preventing duplicate rows on refresh.
- **Lines 5-6** — If no specific contact list was passed in, loads the full list from the database.
- **Lines 8-9** — Inserts one row per contact. Crucially, the Treeview's internal row id (`iid`) is set to the contact's database id (as a string) — this lets later code map a clicked row straight back to its database record without a separate lookup table.
- **Line 11** — Updates a small label showing how many contacts are currently displayed.

`_on_search_changed()` and `_clear_search()`

```
def _on_search_changed(self, *args):
    term = self.search_var.get().strip()
    if term:
        results = self.db.search_contacts(term)
    else:
        results = self.db.get_all_contacts()
    self.refresh_list(results)

def _clear_search(self):
    self.search_var.set("")
```

- **Line 1** — Called automatically on every keystroke in the search box (via the trace set up earlier); `*args` absorbs the extra arguments Tkinter passes to trace callbacks, which are not needed here.
- **Line 2** — Reads the current search box text and removes leading/trailing spaces.
- **Lines 3-6** — If the user has typed something, queries the database's search method; otherwise (empty box) shows every contact.
- **Line 7** — Refreshes the Treeview with the resulting rows — this is what makes the list filter live as the user types, satisfying the 'Search Contact' requirement.
- **Lines 9-10** — The Clear button's handler simply resets the search text, which in turn re-triggers `_on_search_changed` and restores the full list.

_on_select_contact()

```
def _on_select_contact(self, event):
    selection = self.tree.selection()
    if not selection:
        return
    contact_id = int(selection[0])
    contact = self.db.get_contact(contact_id)
    ...
    self.selected_contact_id = contact_id
    _, name, phone, email, address = contact
    self.name_var.set(name)
    self.phone_var.set(phone)
    ...
    self.form_title.config(text=f"Editing: {name}")
    self.save_btn.config(state="disabled")
    self.update_btn.config(state="normal")
```

- **Lines 2-4** — Reads which row(s) are selected in the Treeview; if the selection is empty (for example after a delete), the method exits immediately.
- **Line 5** — Converts the selected row's Treeview id back into an integer database id.
- **Line 6** — Fetches the complete record for that contact from the database.
- **Line 8** — Remembers which contact is loaded, so a later 'Update' click knows which row to modify.
- **Lines 9-11** — Unpacks the database tuple and copies each value into the form's variables, populating the Name, Phone, Email fields (and, further down, the Address text box) with the selected contact's current details.
- **Last three lines** — Updates the form's heading to show which contact is being edited, and swaps the button states — 'Add Contact' is disabled and 'Update Contact' is enabled — so the user cannot accidentally create a duplicate record while editing.

`_validate_form()`

```
def _validate_form(self):
    name = self.name_var.get().strip()
    phone = self.phone_var.get().strip()
    email = self.email_var.get().strip()

    if not name:
        messagebox.showwarning("Validation Error", "Name is required.")
        return None
    if not phone or not self.PHONE_PATTERN.match(phone):
        messagebox.showwarning(...)
        return None
    if email and not self.EMAIL_PATTERN.match(email):
        messagebox.showwarning(...)
        return None

    address = self.address_text.get("1.0", "end").strip()
    return name, phone, email, address
```

- **Lines 2-4** — Reads and trims the current text from each form field.
- **Lines 6-8** — If the name field is empty, shows a warning pop-up and returns None so the caller knows validation failed and should stop.
- **Lines 9-11** — Checks the phone number against a regular expression (PHONE_PATTERN, defined at the top of the class) that only allows digits, spaces, +, -, and parentheses, between 6 and 20 characters.
- **Lines 12-14** — If an email was entered, checks it against a simple EMAIL_PATTERN regular expression requiring the general shape text@text . text; email is optional, so an empty value skips this check.
- **Line 16** — Reads the full multi-line contents of the Address text box, from position '1.0' (line 1, character 0) to 'end'.
- **Line 17** — Returns all four validated values as a tuple, which the calling method unpacks.

save_contact() and update_contact()

```
def save_contact(self):
    data = self._validate_form()
    if not data:
        return
    name, phone, email, address = data
    self.db.add_contact(name, phone, email, address)
    self._show_status(f"Added '{name}' to contacts.")
    self.clear_form()
    self._on_search_changed()

def update_contact(self):
    if self.selected_contact_id is None:
        return
    data = self._validate_form()
    ...
    self.db.update_contact(self.selected_contact_id, name, phone, email, address)
```

- **save_contact, lines 2-4** — Runs validation first; if it failed (returned None), the method exits without touching the database.
- **save_contact, lines 5-6** — Unpacks the validated data and inserts a new row via the database layer.
- **save_contact, lines 7-9** — Shows a temporary success message, clears the form back to 'add' mode, and refreshes the visible list so the new contact appears immediately.
- **update_contact, lines 11-12** — Guards against calling update when no contact is actually selected (the button is disabled in that state, but this is a safety check).
- **update_contact, remaining lines** — Validates the form exactly as save_contact does, then calls the database's update method using the remembered selected_contact_id, and likewise shows a status message, clears the form, and refreshes the list.

delete_selected()

```
def delete_selected(self):
    selection = self.tree.selection()
    if not selection:
        messagebox.showinfo("No Selection", "Please select a contact to delete.")
        return

    contact_id = int(selection[0])
    contact = self.db.get_contact(contact_id)
    name = contact[1] if contact else "this contact"

    confirmed = messagebox.askyesno(
        "Confirm Delete", f"Are you sure you want to delete '{name}'?"
    )
    if confirmed:
        self.db.delete_contact(contact_id)
        self._show_status(f"Deleted '{name}'")
        self.clear_form()
        self._on_search_changed()
```

- **Lines 2-5** — If nothing is selected in the list, shows an informational pop-up and stops — this satisfies the requirement that deleting always requires an explicit selection.
- **Lines 7-9** — Looks up the selected contact's name so it can be shown in the confirmation dialog, falling back to a generic phrase if the record was somehow already gone.
- **Lines 11-13** — Displays a Yes/No confirmation dialog before deleting anything, preventing accidental data loss.
- **Lines 14-18** — Only if the user confirms: deletes the row from the database, shows a status message, resets the form, and refreshes the visible list.

clear_form(), _show_status(), and on_close()

```
def clear_form(self):
    self.selected_contact_id = None
    self.name_var.set("")
    ...
    self.form_title.config(text="Add New Contact")
    self.save_btn.config(state="normal")
    self.update_btn.config(state="disabled")

def _show_status(self, message):
    self.status_label.config(text=message)
    self.root.after(3500, lambda: self.status_label.config(text=''))

def on_close(self):
    self.db.close()
    self.root.destroy()
```

- **clear_form** — Resets every form field to empty, clears the remembered selection, restores the heading to 'Add New Contact', and flips the button states back to 'add' mode (Save enabled, Update disabled).
- **_show_status** — Displays a short confirmation message (e.g. 'Added ... to contacts.') and uses `root.after(3500, ...)` to automatically clear that message after 3.5 seconds, so old messages don't linger on screen.
- **on_close** — Runs when the window's close button is pressed: closes the SQLite connection cleanly before destroying the Tkinter window, avoiding leaving the database file in a locked state.

5.5 Program Entry Point

main() and the __main__ guard

```
def main():
    root = tk.Tk()
    app = ContactBookApp(root)
    root.protocol("WM_DELETE_WINDOW", app.on_close)
    root.mainloop()

if __name__ == "__main__":
    main()
```

- **Line 2** — Creates the main Tkinter application window.
- **Line 3** — Instantiates the whole application, which builds the interface and loads existing contacts.
- **Line 4** — Overrides the window manager's default close behaviour so that clicking the window's 'X' button calls `app.on_close` instead of closing abruptly — ensuring the database connection is closed properly.
- **Line 5** — Starts Tkinter's event loop, which keeps the window open and responsive to clicks, keystrokes, and other events until the program exits.
- **Final two lines** — The standard Python idiom that runs `main()` only when this file is executed directly (e.g. `python contact_book.py`), not when it is imported as a module elsewhere.

6. Feature-to-Code Mapping

The table below maps each originally requested feature to the exact part of the code responsible for it.

Requirement	Implemented By
Store name, phone, email, address	<code>contacts</code> SQLite table (Section 4.1) and <code>ContactDatabase._create_table()</code>
Add Contact	<code>ContactDatabase.add_contact()</code> and <code>ContactBookApp.save_contact()</code>
View Contact List	<code>ContactDatabase.get_all_contacts()</code> and <code>ContactBookApp.refresh_list()</code> / Treeview widget
Search Contact	<code>ContactDatabase.search_contacts()</code> and <code>ContactBookApp._on_search_changed()</code>
Update Contact	<code>ContactDatabase.update_contact()</code> and <code>ContactBookApp.update_contact()</code>
Delete Contact	<code>ContactDatabase.delete_contact()</code> and <code>ContactBookApp.delete_selected()</code>
User-friendly interface	<code>_build_style()</code> , <code>_build_layout()</code> , <code>_build_list_panel()</code> , <code>_build_form_panel()</code>

7. Testing Summary

- Adding a contact with all fields filled correctly saves and appears in the list.
- Adding a contact with an empty name or invalid phone number is blocked with a warning.
- Searching by a partial name or partial phone number correctly filters the list live.
- Selecting a contact from the list correctly loads its details into the edit form.
- Updating a selected contact's details correctly overwrites the existing record.
- Deleting a contact prompts for confirmation and removes the record only after confirmation.
- Closing and reopening the application correctly reloads all previously saved contacts.

8. Conclusion

The Contact Book project successfully demonstrates the integration of a Tkinter-based graphical interface with a persistent SQLite backend to deliver a complete, practical CRUD application. Every requirement — storing contact information, adding, viewing, searching, updating, and deleting contacts, all through a user-friendly interface — has been implemented and verified. The clean separation between the `ContactDatabase` and `ContactBookApp` classes also makes the codebase easy to extend or maintain in the future.

9. Future Enhancements

- Add contact photos and group/category tagging (e.g. Family, Work, Friends).
- Add import/export support for CSV or vCard (.vcf) files.
- Add sorting options (by name, most recently added, etc.) in the contact list.
- Add a dark-mode theme toggle.
- Package the application as a standalone executable using PyInstaller for easier distribution.